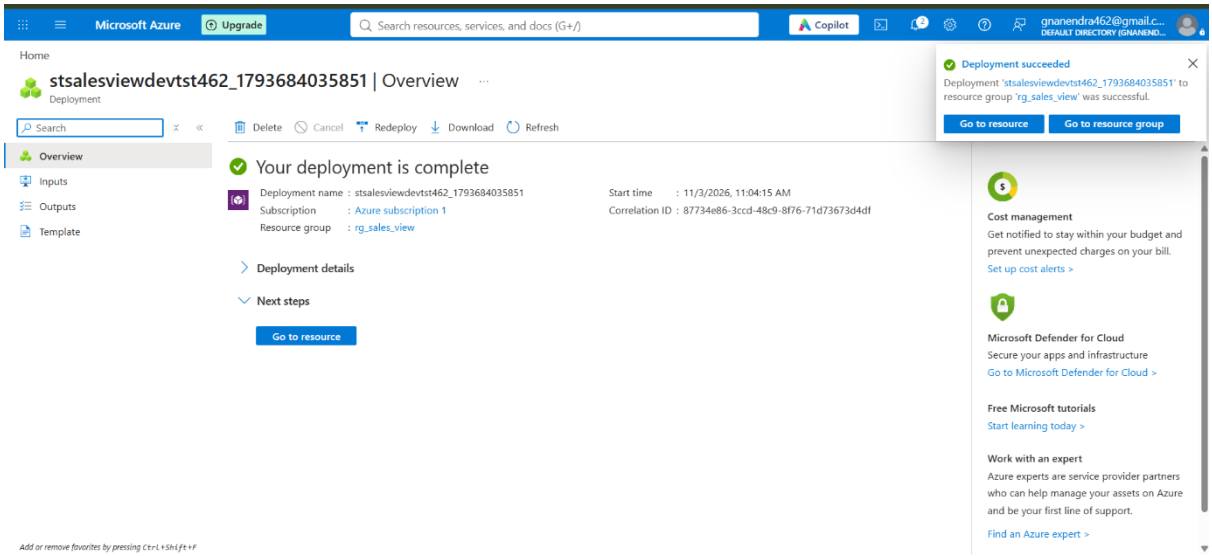
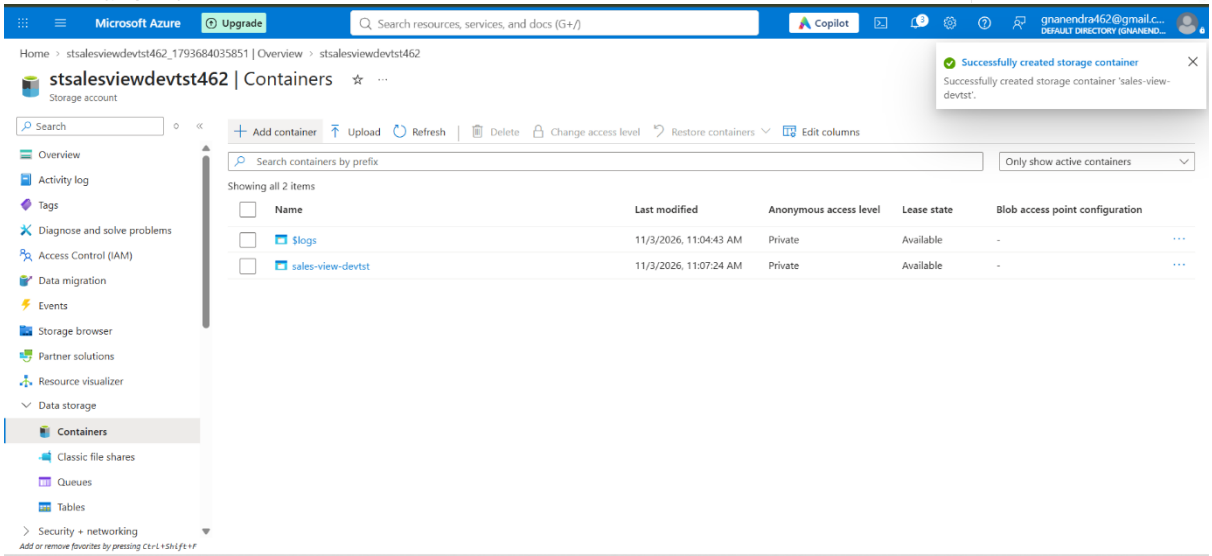
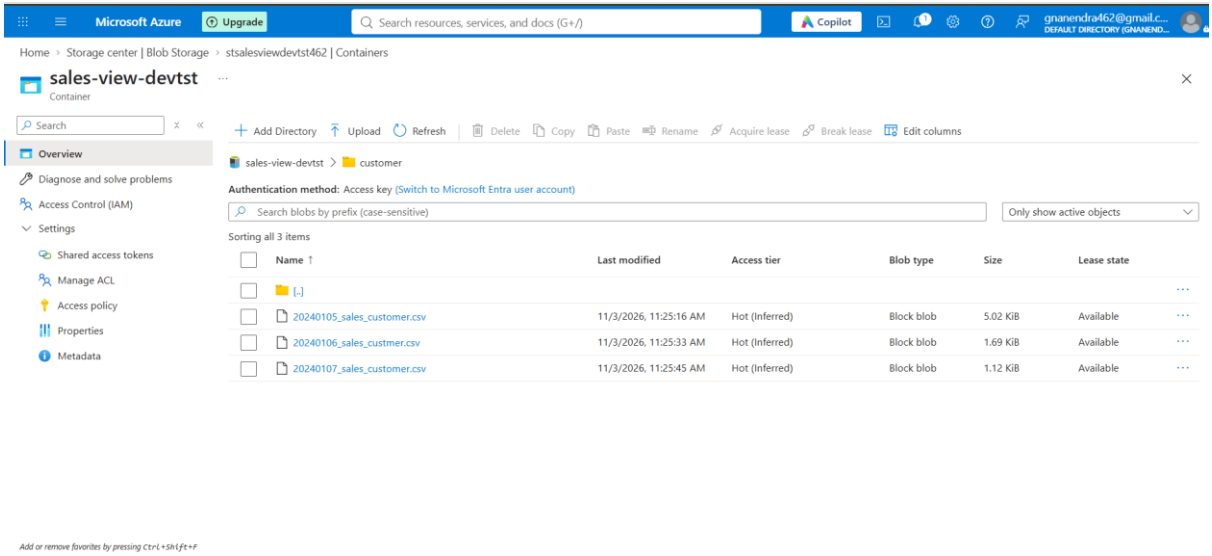
ADF ASSIGNMENT

1) Create a container in ADLS with the project name as sales_view_devtst

- Create a folder for customer, product, store, and sales - Upload the files in sequence to make it work in real-time.

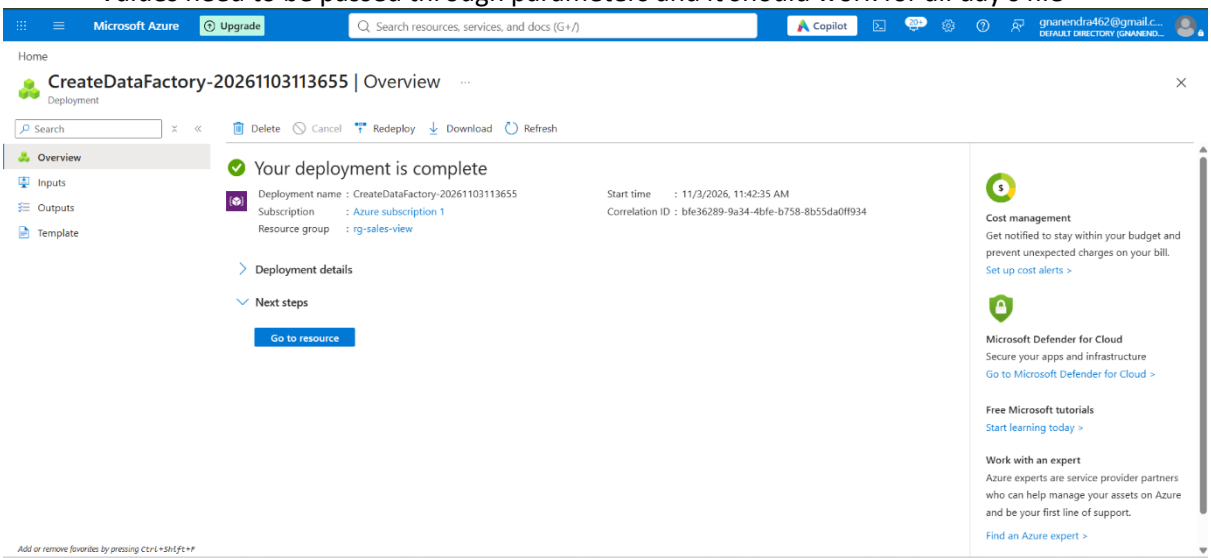


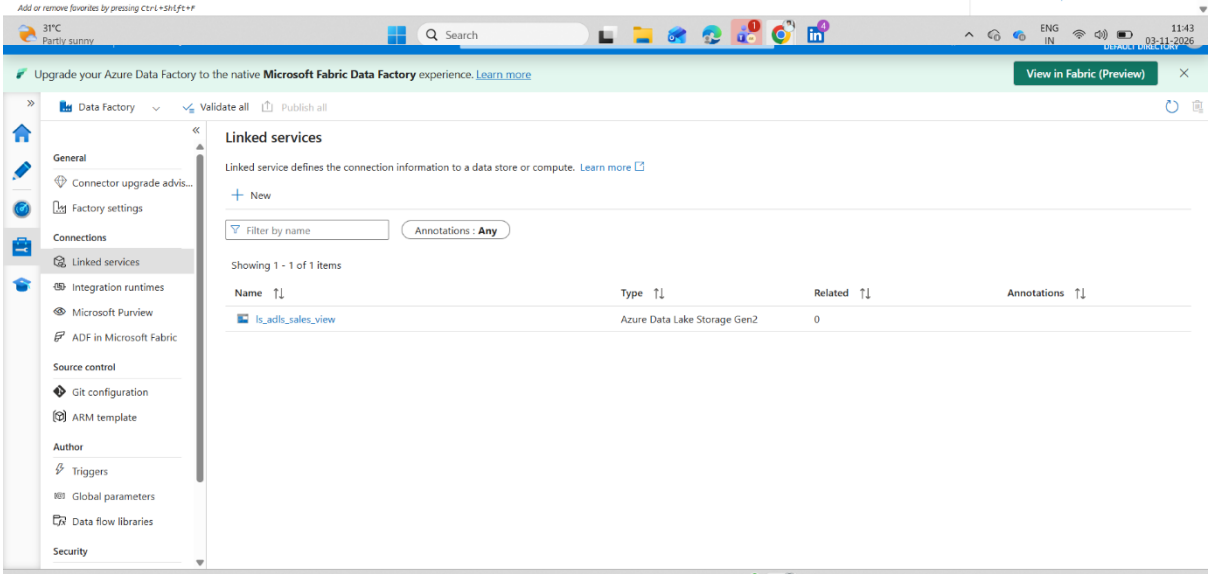


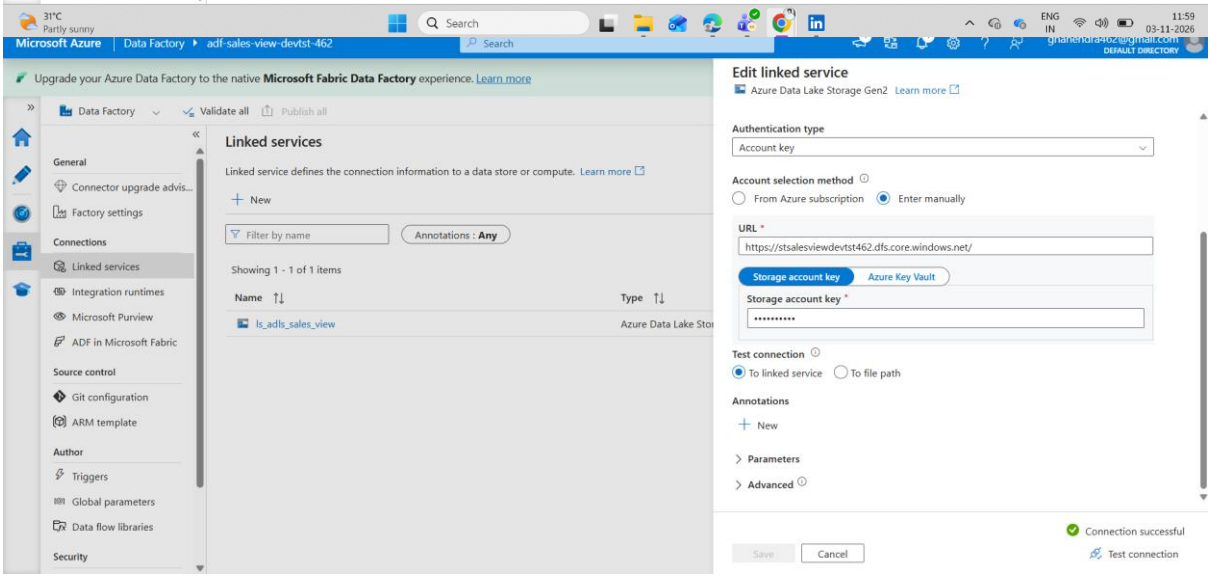


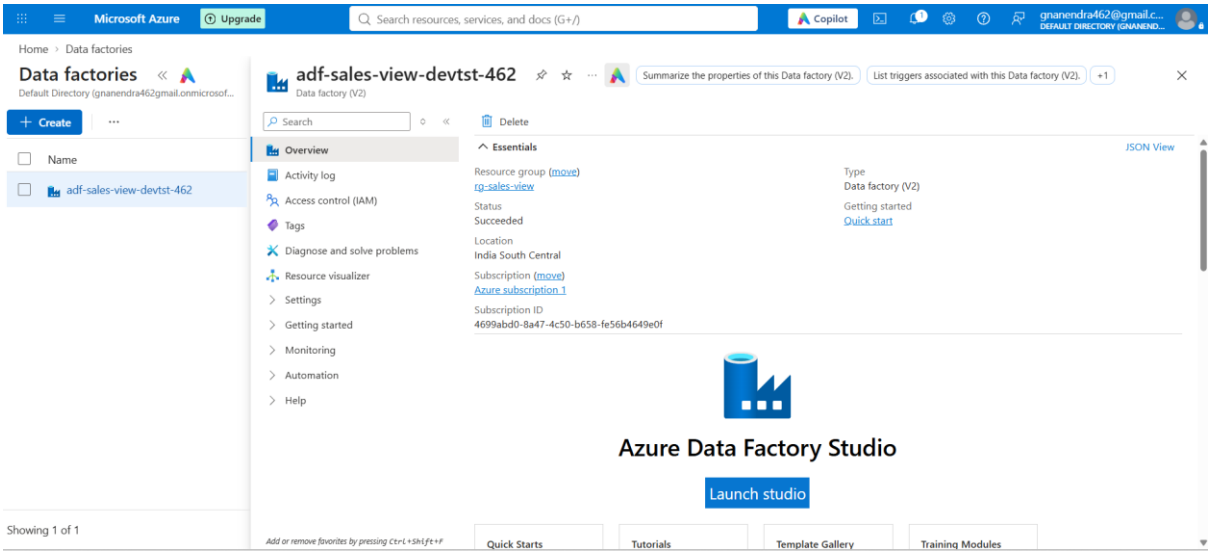
2) Create an ADF Pipeline to get the latest modified files the folders of the ADLS

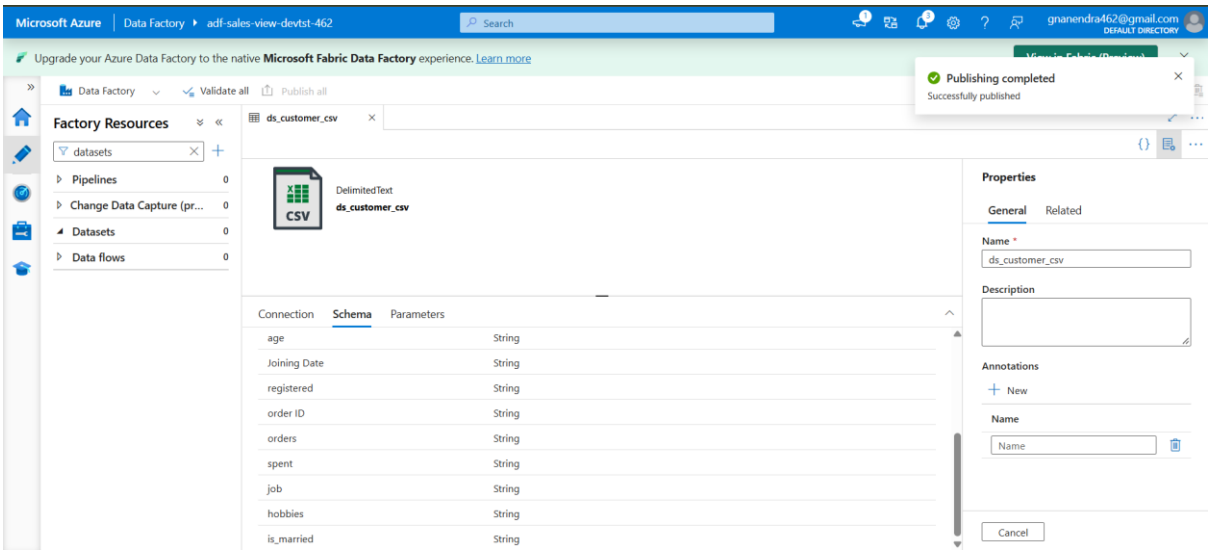
- Parameterize the Pipeline to work dynamically.
- Values need to be passed through parameters and it should work for all day's file

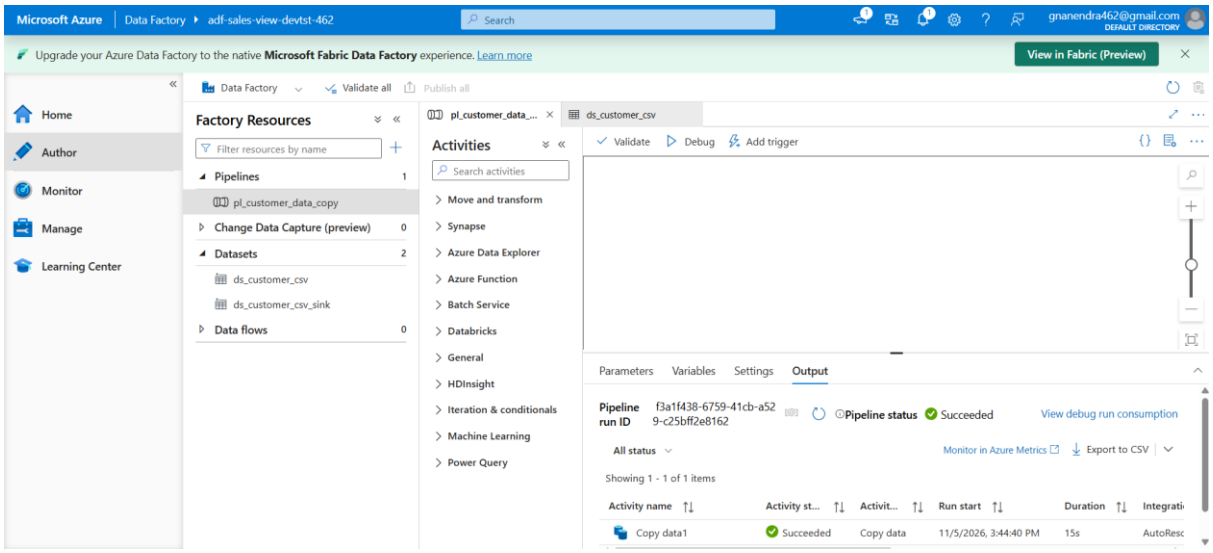


- 

- 


Microsoft Azure
Upgrade
Search resources, services, and docs (G+/)
Copilot
Home
Data factories
Data factories
Default Directory (gnanendra462gmail.onmicrosof...
Create
Name
adf-sales-view-devtst-462
adf-sales-view-devtst-462
Data factory (V2)
Summarize the properties of this Data factory (V2).
List triggers associated with this Data factory (V2).
Search
Delete
Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Resource visualizer
Settings
Getting started
Monitoring
Automation
Help
Essentials
JSON View
Resource group (move)
rg-sales-view
Status
Succeeded
Location
India South Central
Subscription (move)
Azure subscription 1
Subscription ID
4699abd0-8a47-4c50-b658-fe56b4649e0f
Type
Data factory (V2)
Getting started
Quick start
Azure Data Factory Studio
Launch studio
Showing 1 of 1
Quick Starts
Tutorials
Template Gallery
Training Modules

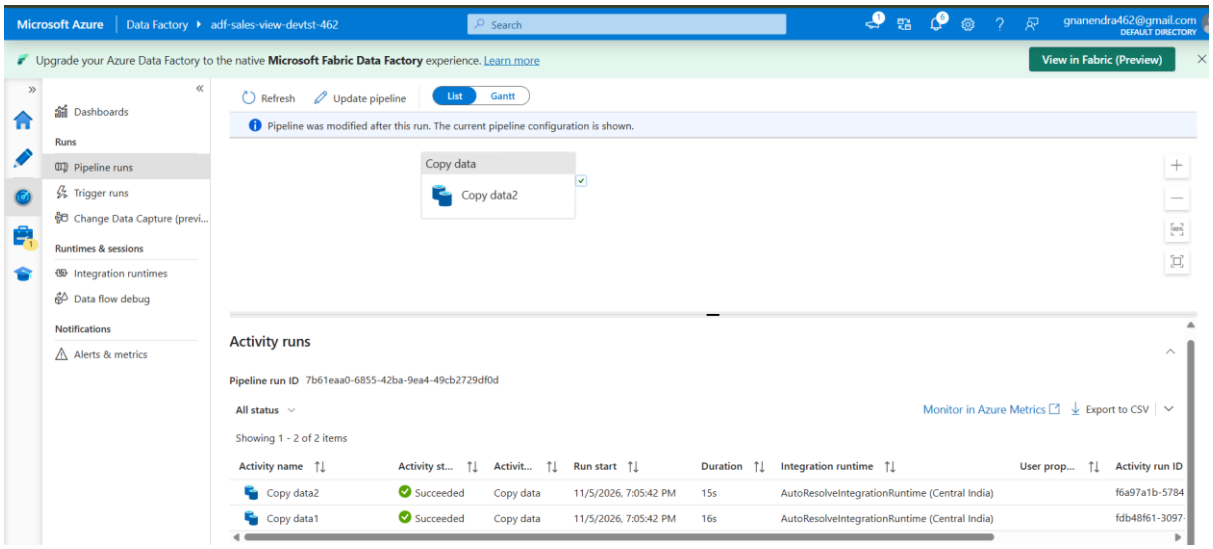

Microsoft Azure
Data Factory
adf-sales-view-devtst-462
Search
gnanendra462@gmail.com
DEFAULT DIRECTORY
Upgrade your Azure Data Factory to the native Microsoft Fabric Data Factory experience. Learn more
Publishing completed
Successfully published
Data Factory
Validate all
Publish all
Factory Resources
datasets
Pipelines
Change Data Capture (pr...
Datasets
Data flows
ds_customer_csv
DelimitedText
ds_customer_csv
Connection
Schema
Parameters
age String
Joining Date String
registered String
order ID String
orders String
spent String
job String
hobbies String
is_married String
Properties
General
Related
Name *
ds_customer_csv
Description
Annotations
New
Name
Name
Cancel


Microsoft Azure
Data Factory
adf-sales-view-devtst-462
Search
gnanendra462@gmail.com
DEFAULT DIRECTORY
Upgrade your Azure Data Factory to the native Microsoft Fabric Data Factory experience. Learn more
View in Fabric (Preview)
Data Factory
Validate all
Publish all
Home
Author
Monitor
Manage
Learning Center
Factory Resources
Filter resources by name
Pipelines 1
pl_customer_data_copy
Change Data Capture (preview) 0
Datasets 2
ds_customer_csv
ds_customer_csv_sink
Data flows 0
pl_customer_data_...
ds_customer_csv
Activities
Search activities
Move and transform
Synapse
Azure Data Explorer
Azure Function
Batch Service
Databricks
General
HDInsight
Iteration & conditionals
Machine Learning
Power Query
Validate
Debug
Add trigger
Parameters
Variables
Settings
Output
Pipeline run ID
f3a1f438-6759-41cb-a529-c25bff2e8162
Pipeline status
Succeeded
View debug run consumption
All status
Monitor in Azure Metrics
Export to CSV
Showing 1 - 1 of 1 items
Activity name
Activity st...
Activit...
Run start
Duration
Integrati
Copy data1
Succeeded
Copy data
11/5/2026, 3:44:40 PM
15s
AutoResc

3) Create a Bronze/sales view/ (Bronze – container)

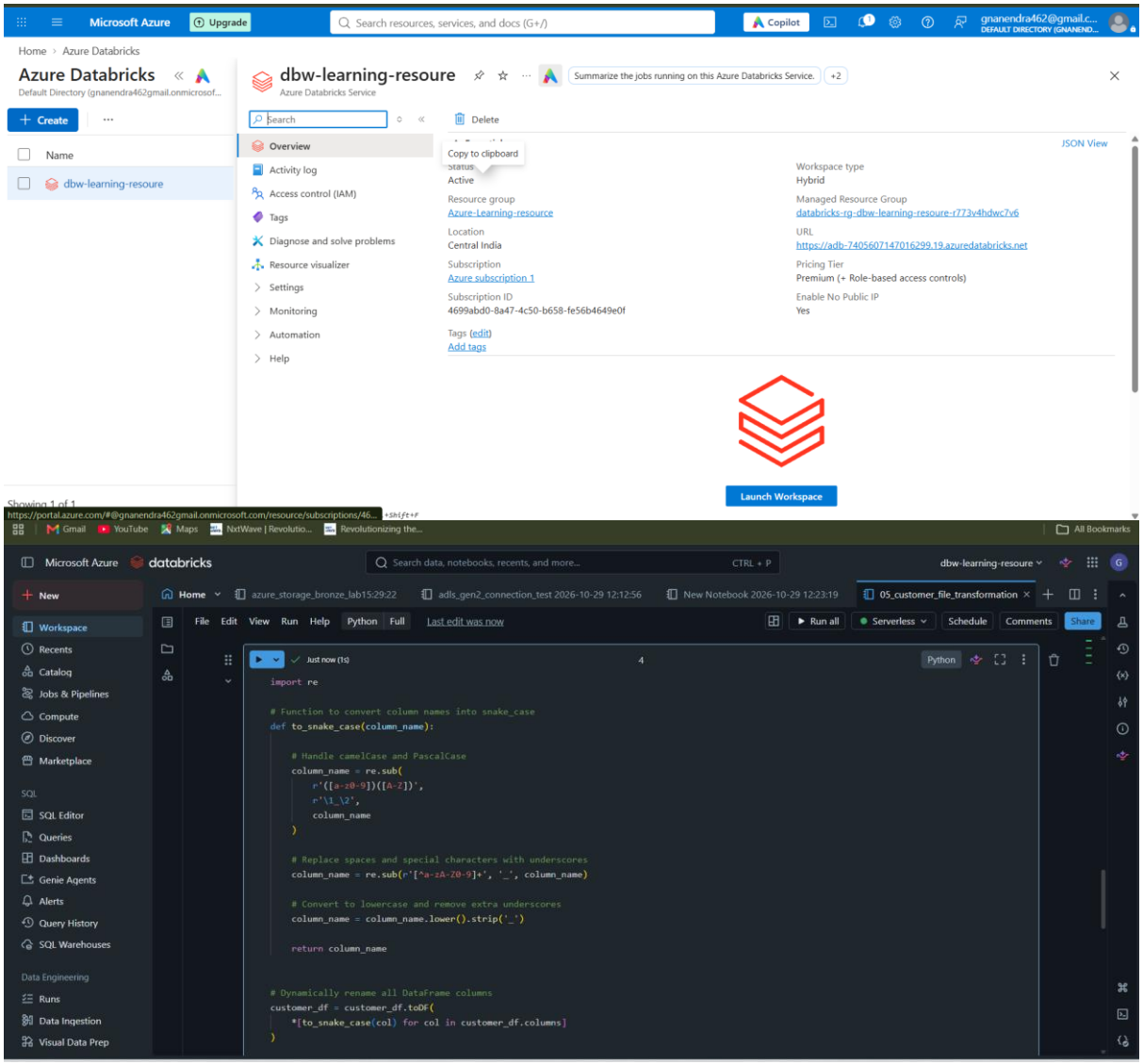
- Subfolders-> customer, product, store, sales, and store the raw data copied from the ADF p`ipeline

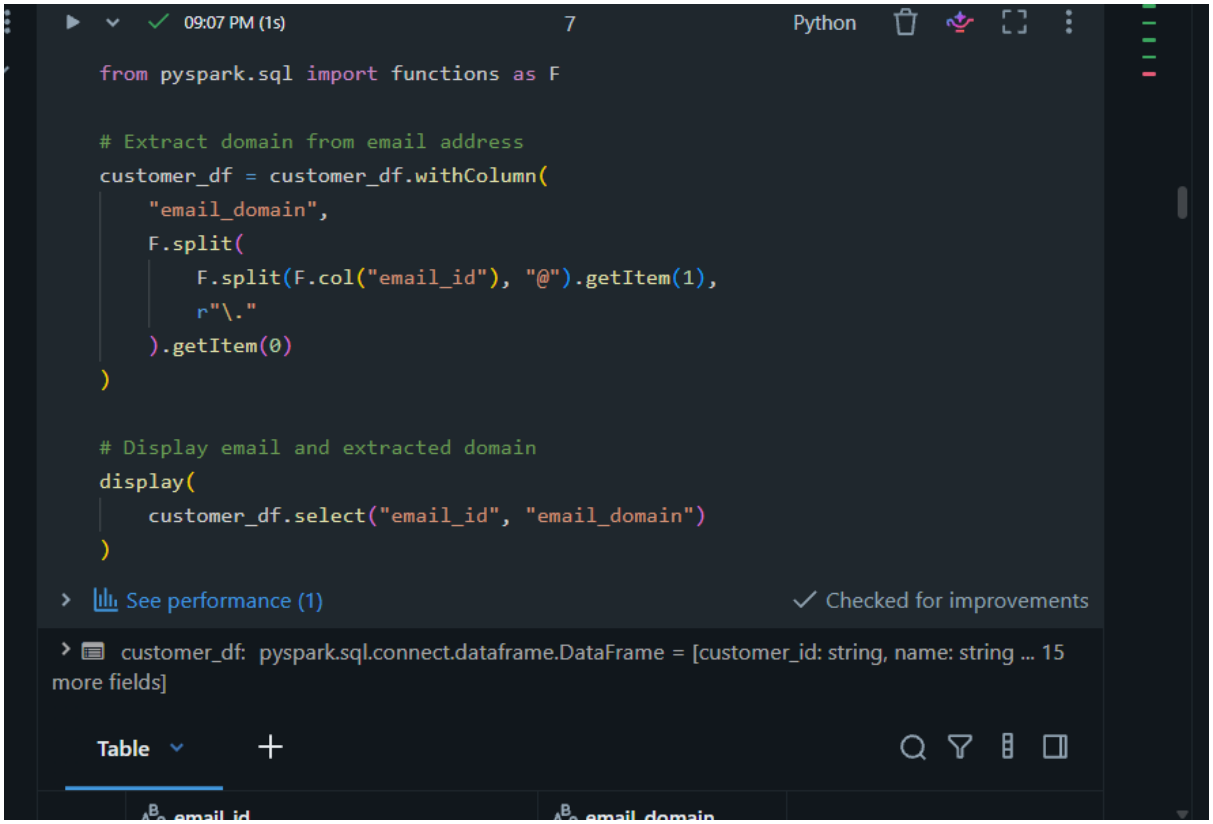
Database name: sales_view

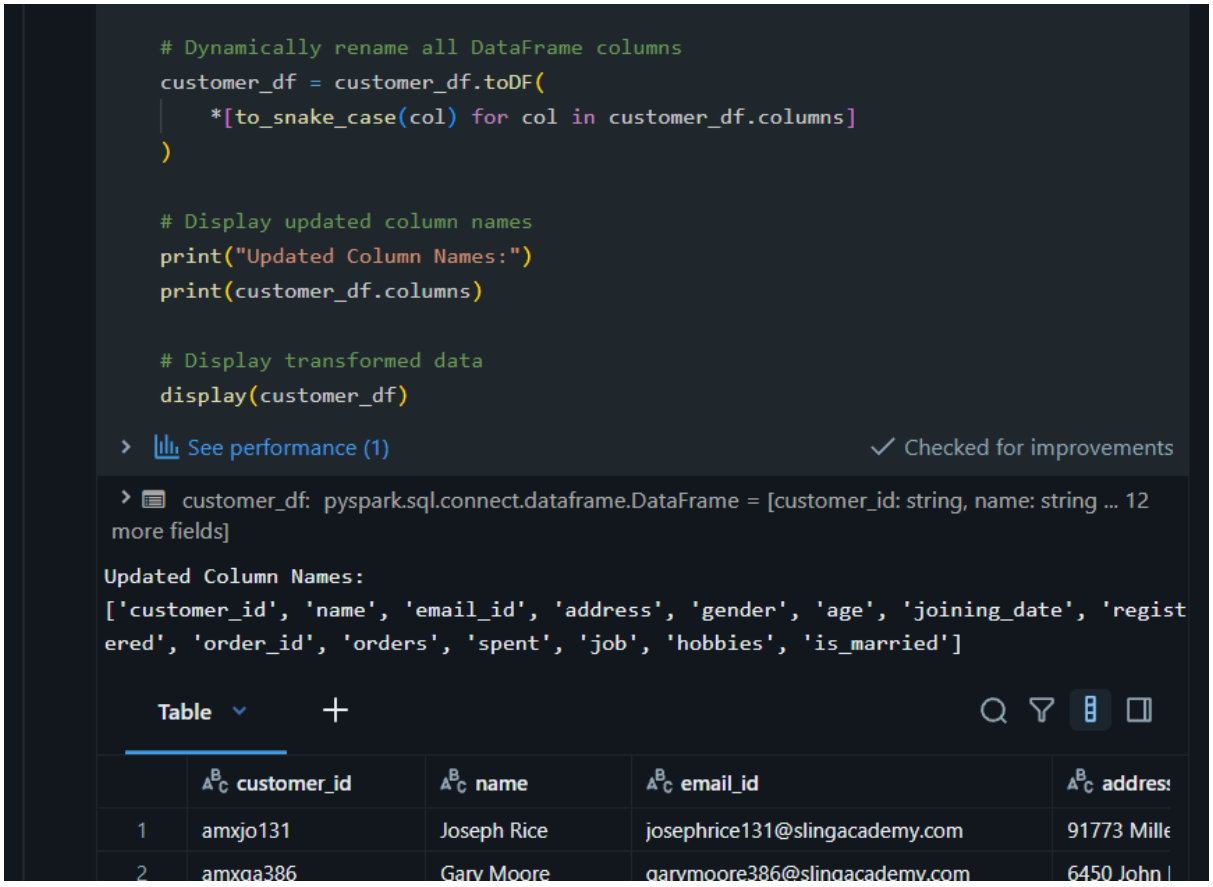


4) Customer file

- All the column headers should be in snake case in lower case (By using UDF function dynamically works for all camel case to snake case)
- By using the "Name" column split by " " and create two columns first_name and last_name
- Create column domain and extract from email columns Ex: Email = "josephrice131@slingacademy.com" domain="slingacademy"
- Create a column ‘gender where male = "M" and Female="F"
- From Joining date create two columns date and time by splitting based on " " delimiter.
- Date column should be on "yyyy-MM-dd" format.
- Create a column expenditure-status, based on spent column is spent below 200 column value is "MINIMUM" else "MAXIMUM"
- Write based on upsert [table_name: customer] (in silver layer path is silver/sales_view/tablename/{delta pearquet}


Microsoft Azure
Upgrade
Search resources, services, and docs (G+/)
Copilot
gnanendra462@gmail.c...
DEFAULT DIRECTORY (GNANEND...
Home > Azure Databricks
Azure Databricks
Default Directory (gnanendra462gmail.onmicrosof...
Create
Name
dbw-learning-resoure
dbw-learning-resoure
Azure Databricks Service
Summarize the jobs running on this Azure Databricks Service.
+2
Search
Delete
Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Resource visualizer
Settings
Monitoring
Automation
Help
Copy to clipboard
JSON View
Status
Active
Workspace type
Hybrid
Resource group
Azure-Learning-resource
Managed Resource Group
databricks-rg-dbw-learning-resoure-r773v4hdwc7v6
Location
Central India
URL
https://adb-7405607147016299.19.azuredatabricks.net
Subscription
Azure subscription 1
Pricing Tier
Premium (+ Role-based access controls)
Subscription ID
4699abd0-8a47-4c50-b658-fe56b4649e0f
Enable No Public IP
Yes
Tags (edit)
Add tags
Launch Workspace
Showing 1 of 1
Gmail
YouTube
Maps
NxtWave | Revolutio...
Revolutionizing the...
All Bookmarks
Microsoft Azure
databricks
Search data, notebooks, recents, and more...
CTRL + P
dbw-learning-resoure
New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Discover
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie Agents
Alerts
Query History
SQL Warehouses
Data Engineering
Runs
Data Ingestion
Visual Data Prep
Home
azure_storage_bronze_lab15:29:22
adls_gen2_connection_test 2026-10-29 12:12:56
New Notebook 2026-10-29 12:23:19
05_customer_file_transformation
File Edit View Run Help Python Full Last edit was now
Run all
Serverless
Schedule
Comments
Share
Just now (1s)
4
Python
import re
Function to convert column names into snake_case
def to_snake_case(column_name):
Handle camelCase and PascalCase
column_name = re.sub(
r'([a-z0-9])([A-Z])',
r'\1_\2',
column_name
)
Replace spaces and special characters with underscores
column_name = re.sub(r'[^a-zA-Z0-9]+', '_', column_name)
Convert to lowercase and remove extra underscores
column_name = column_name.lower().strip('_')
return column_name
Dynamically rename all DataFrame columns
customer_df = customer_df.toDF(
*[to_snake_case(col) for col in customer_df.columns]
)


09:07 PM (1s)
7
Python
from pyspark.sql import functions as F
Extract domain from email address
customer_df = customer_df.withColumn(
"email_domain",
F.split(
F.split(F.col("email_id"), "@").getItem(1),
r"\."
).getItem(0)
)
Display email and extracted domain
display(
customer_df.select("email_id", "email_domain")
)
See performance (1)
Checked for improvements
customer_df: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, name: string ... 15 more fields]
Table


Dynamically rename all DataFrame columns
customer_df = customer_df.toDF(
*[to_snake_case(col) for col in customer_df.columns]
)
Display updated column names
print("Updated Column Names:")
print(customer_df.columns)
Display transformed data
display(customer_df)
See performance (1)
Checked for improvements
customer_df: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, name: string ... 12 more fields]
Updated Column Names:
['customer_id', 'name', 'email_id', 'address', 'gender', 'age', 'joining_date', 'registered', 'order_id', 'orders', 'spent', 'job', 'hobbies', 'is_married']
Table
customer_id
name
email_id
address
1
amxjo131
Joseph Rice
josephrice131@slingacademy.com
91773 Mille
2
amxga386
Gary Moore
garymoore386@slingacademy.com
6450 John

5) Product File

- All the column header shlould be in snake case in lower case (use same UDF Function)
- Create a column sub_category (Use Category columns id category_id=1, "phone"; 2, "laptop"; 3,"playstation"; 4,"e-device"
- Write based on upsert [table_name: product](in silver layer path is silver/sales_view/tablename/{delta pearquet}

```
volume_path = "/Volumes/dbw_learning_resoure/default/customer_data_volume/"

# List only product CSV files
product_files = [
    file.path
    for file in dbutils.fs.ls(volume_path)
    if "sales_product" in file.name
]

# Display product file paths
for file_path in product_files:
    print(file_path)
```

See performance (1)

```
dbfs:/Volumes/dbw_learning_resoure/default/customer_data_volume/20240105_sales_product.csv
dbfs:/Volumes/dbw_learning_resoure/default/customer_data_volume/20240106_sales_product.csv
dbfs:/Volumes/dbw_learning_resoure/default/customer_data_volume/20240107_sales_product.csv
```



```
        "product_id column not found. "
        "Check the actual product primary key before upserting."
    )

# Remove duplicate product IDs within this batch
product_df = product_df.dropDuplicates(["product_id"])

if DeltaTable.isDeltaTable(spark, silver_path):

    # Existing Delta table: perform UPSERT
    delta_table = DeltaTable.forPath(spark, silver_path)

    (
        delta_table.alias("target")
        .merge(
            product_df.alias("source"),
            "target.product_id = source.product_id"
        )
        .whenMatchedUpdateAll()
        .whenNotMatchedInsertAll()
        .execute()
    )

    print("Product Silver Table Upsert Completed.")
```

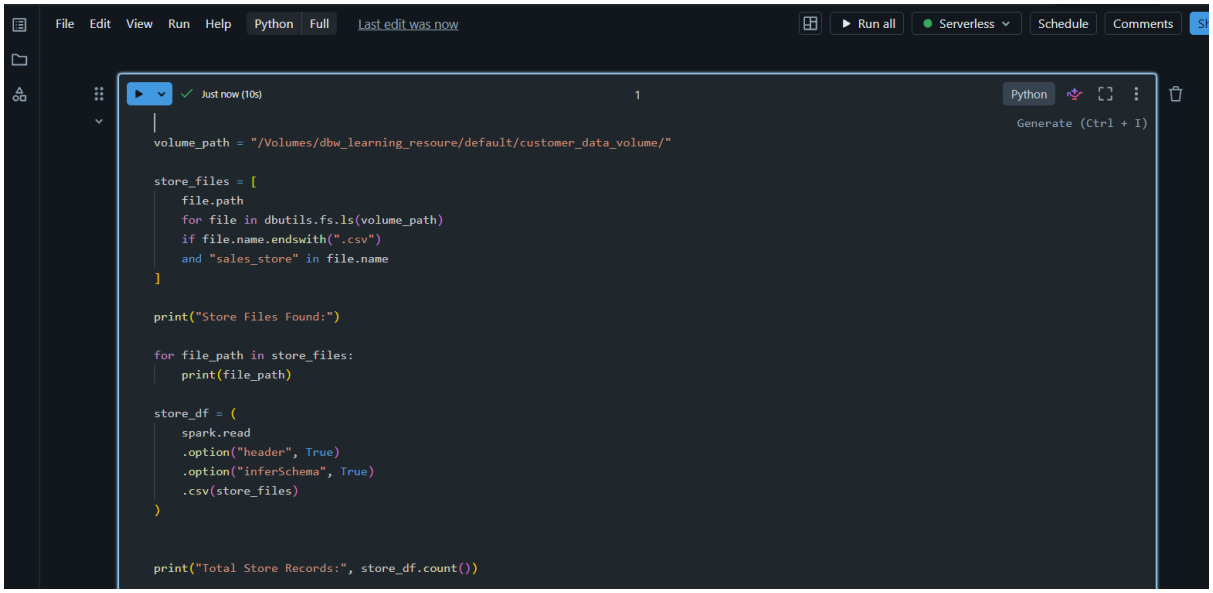
```
dbfs:/Volumes/dbw_learning_resoure/default/customer_data_volume/20240105_sales_product.
csv
dbfs:/Volumes/dbw_learning_resoure/default/customer_data_volume/20240106_sales_product.
csv
dbfs:/Volumes/dbw_learning_resoure/default/customer_data_volume/20240107_sales_product.
csv
Total Product Records: 16
```

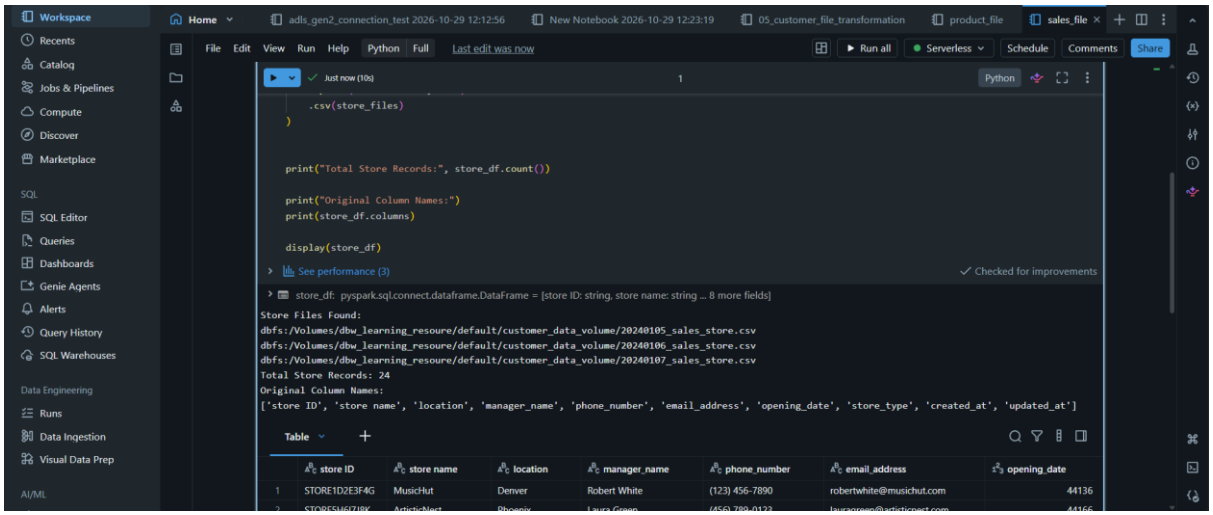
Table +

	product ID	product name	product code	description
1	PROD1A23B4	Apple iPhone 12	IPHN12	The latest iPhone mod
2	PROD5C67D8E	Samsung Galaxy S21	SGS21	High-performance And
3	PRODA9B1C2D3	HP Pavilion Laptop	HPVPLP	Powerful laptop for wo
4	PROD4E5F6G7H	Dell 27-inch Monitor	DELLMON	High-resolution monit
5	PROD2I3J4K5L	Sony PlayStation 5	PS5	Next-gen gaming cons
6	PROD6M7N8O9P	Nintendo Switch	NSW	Hybrid gaming consol
7	PROD4Q5R6S7T	Amazon Echo Dot (4th Ge...	ECHO4	Smart speaker with Ale
8	PROD8U9V1W2	Canon EOS Rebel T7i	REBELT7I	DSLR camera for phot

6) Store

- Read the data make sure header shlould be in snake case in lower case (use same UDF Function)
- Create a store category columns and the value is exatracted from email Eg: "electromart" from johndoe@electromart.com
- created_at, updated_at date as yyyy-MM-dd format
- Write based on upsert [table_name: store] (in silver layer path is silver/sales_view/tablename/{delta pearquet}





```python
# Convert all Store column names
store_df = store_df.toDF(
    *[to_snake_case(col) for col in store_df.columns]
)

# Print updated column names
print("Updated Store Column Names:")
print(store_df.columns)

# Display Store DataFrame
display(store_df)
```

See performance (1) ✓ Checked for improvements

store_df: pyspark.sql.connect.dataframe.DataFrame = [store id: string, store name: string ... 8 more fields]

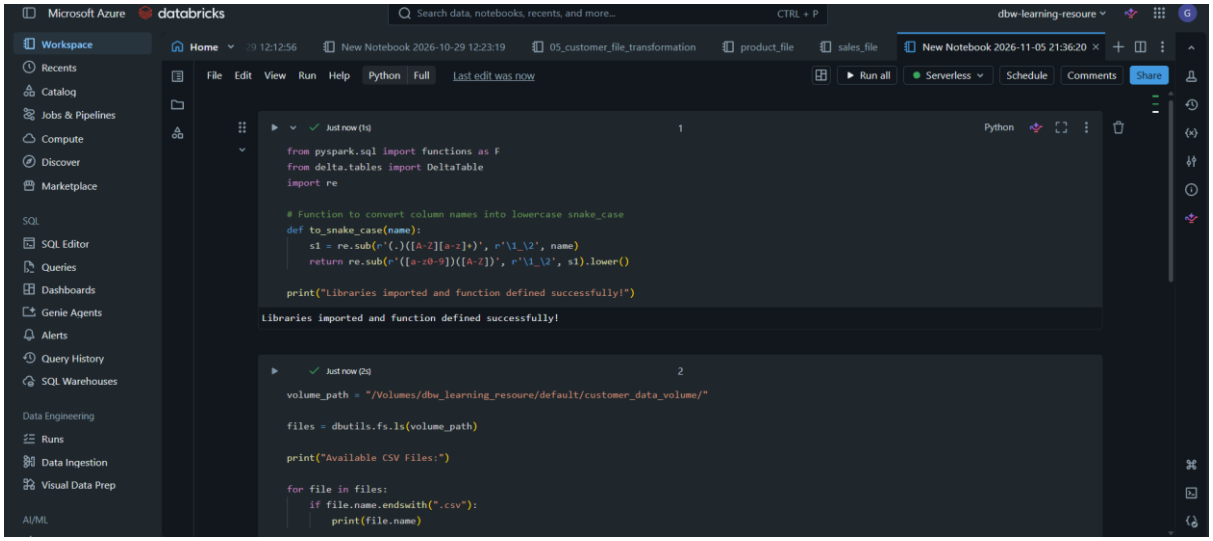
```
Updated Store Column Names:
['store id', 'store name', 'location', 'manager_name', 'phone_number', 'email_address', 'opening_date', 'store_type', 'created_at', 'updated_at']
```

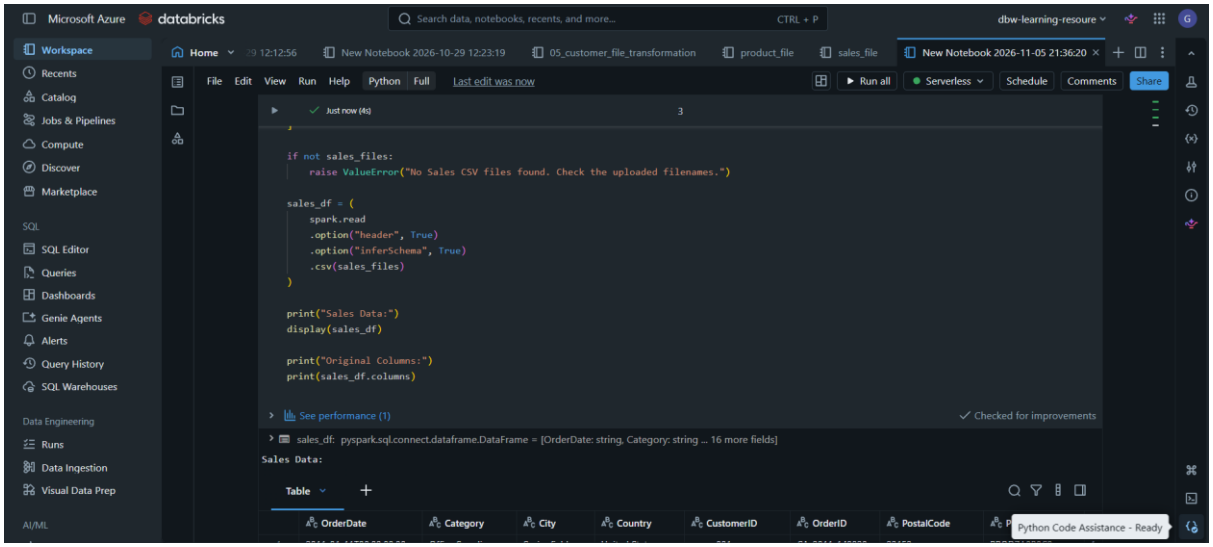
Table +

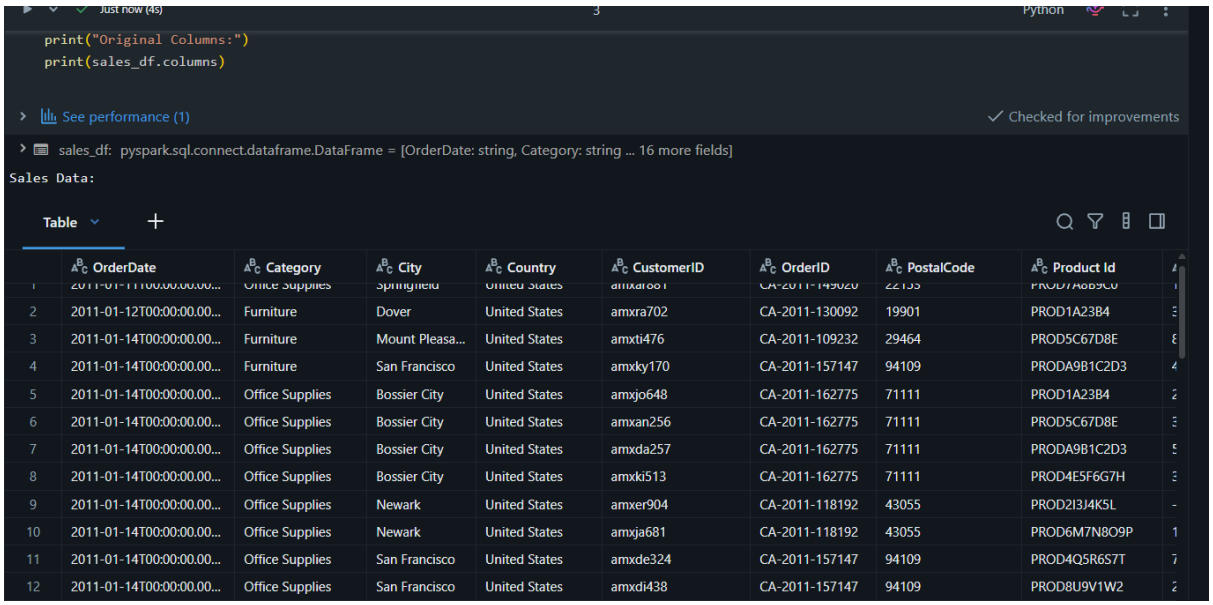
	store id	store name	location	manager_name	phone_number	email_address	opening_date
1	STORE1D2E3F4G	MusicHut	Denver	Robert White	(123) 456-7890	robertwhite@musichut.com	44136
2	STORE5H6I7J8K	ArtisticNest	Phoenix	Laura Green	(456) 789-0123	lauragreen@artisticnest.com	44166
3	STORE1L9M0N1O2	OutdoorLife	Portland	Steven Gray	(789) 012-3456	stevengray@outdoorlife.com	44197

8) Sales

- Read the data make sure header shlould be in snake case in lower case (use same UDF Function)
- Write based on upsert [table_name: customer_sales] (in silver layer path is silver/sales_view/tablename/{delta pearquet}



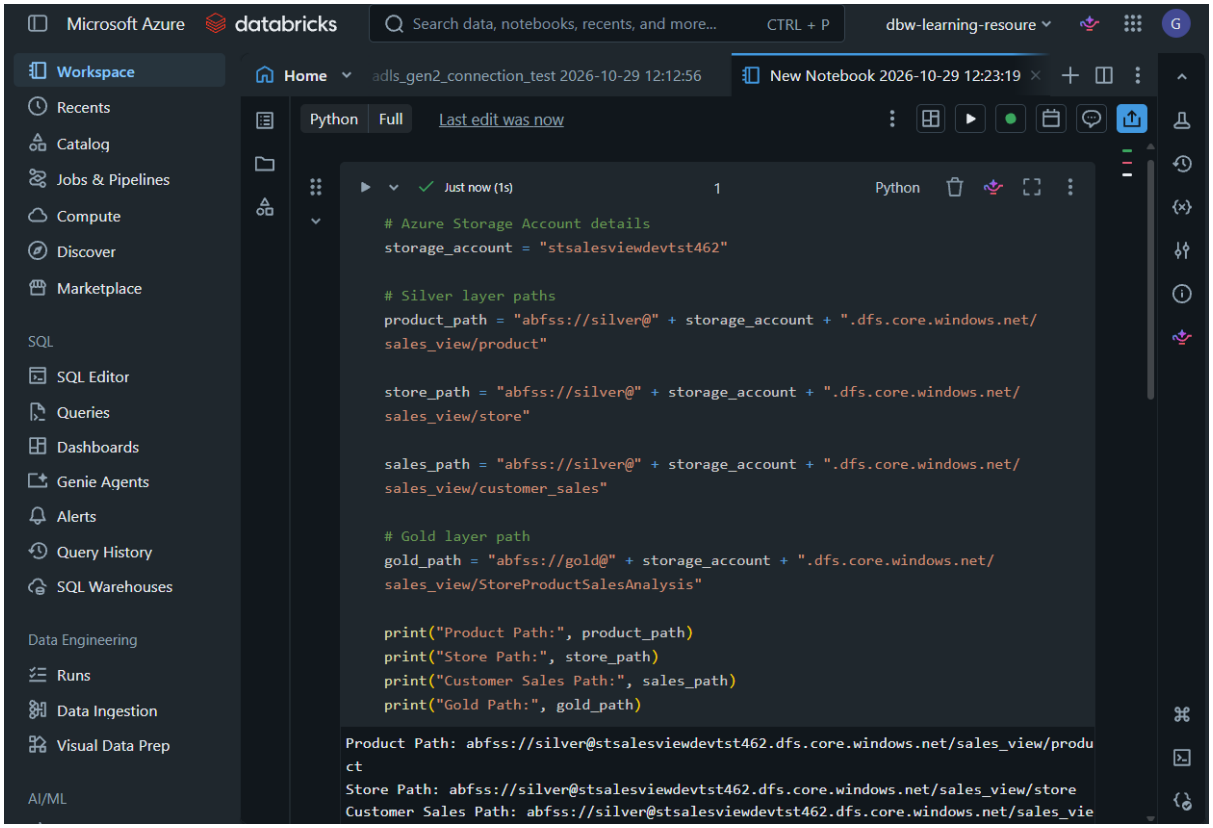


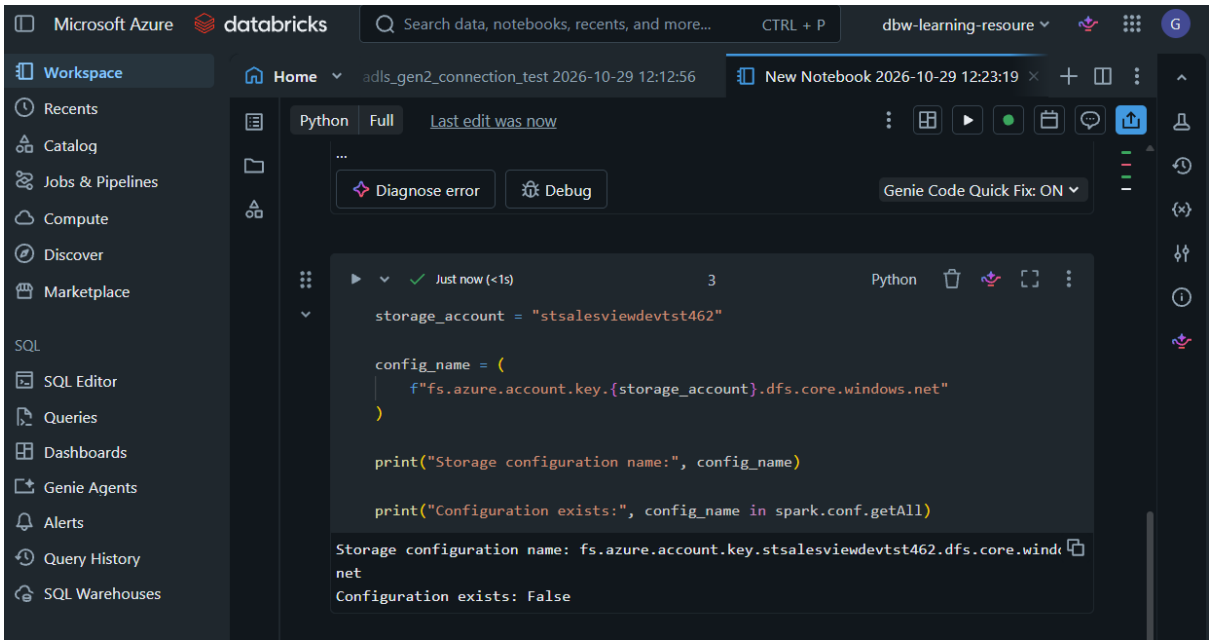


	OrderDate	Category	City	Country	CustomerID	OrderID	PostalCode	Product Id
2	2011-01-12T00:00:00.00...	Furniture	Dover	United States	amxra702	CA-2011-130092	19901	PROD1A23B4
3	2011-01-14T00:00:00.00...	Furniture	Mount Pleasa...	United States	amxti476	CA-2011-109232	29464	PROD5C67D8E
4	2011-01-14T00:00:00.00...	Furniture	San Francisco	United States	amxky170	CA-2011-157147	94109	PRODA9B1C2D3
5	2011-01-14T00:00:00.00...	Office Supplies	Bossier City	United States	amxjo648	CA-2011-162775	71111	PROD1A23B4
6	2011-01-14T00:00:00.00...	Office Supplies	Bossier City	United States	amxan256	CA-2011-162775	71111	PROD5C67D8E
7	2011-01-14T00:00:00.00...	Office Supplies	Bossier City	United States	amxda257	CA-2011-162775	71111	PRODA9B1C2D3
8	2011-01-14T00:00:00.00...	Office Supplies	Bossier City	United States	amxki513	CA-2011-162775	71111	PROD4E5F6G7H
9	2011-01-14T00:00:00.00...	Office Supplies	Newark	United States	amxer904	CA-2011-118192	43055	PROD2I3J4K5L
10	2011-01-14T00:00:00.00...	Office Supplies	Newark	United States	amxja681	CA-2011-118192	43055	PROD6M7N8O9P
11	2011-01-14T00:00:00.00...	Office Supplies	San Francisco	United States	amxde324	CA-2011-157147	94109	PROD4Q5R6S7T
12	2011-01-14T00:00:00.00...	Office Supplies	San Francisco	United States	amxdi438	CA-2011-157147	94109	PROD8U9V1W2

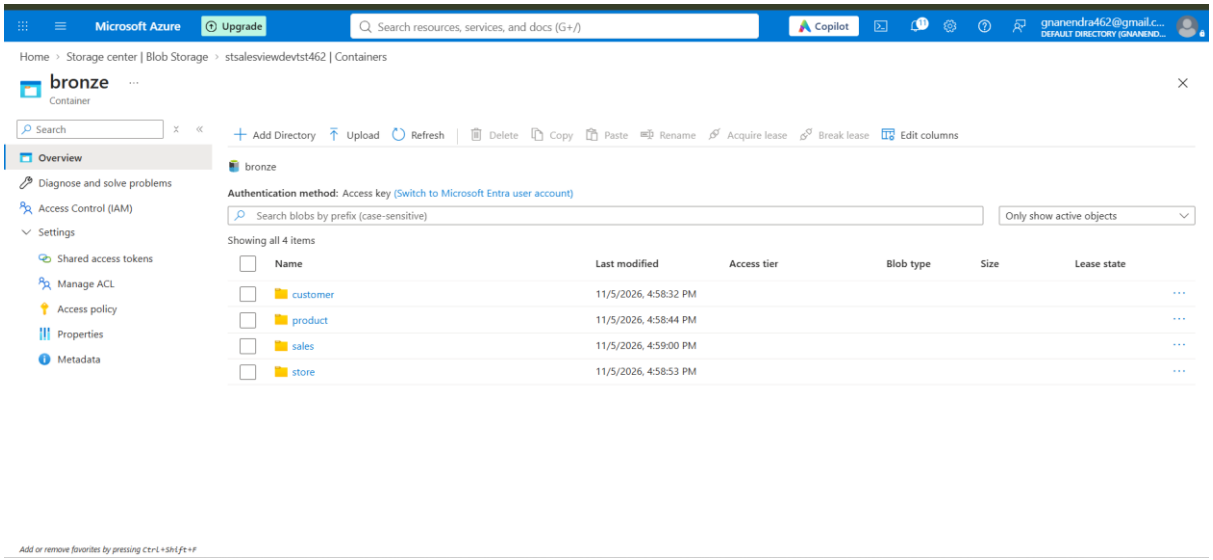
8) In gold layer

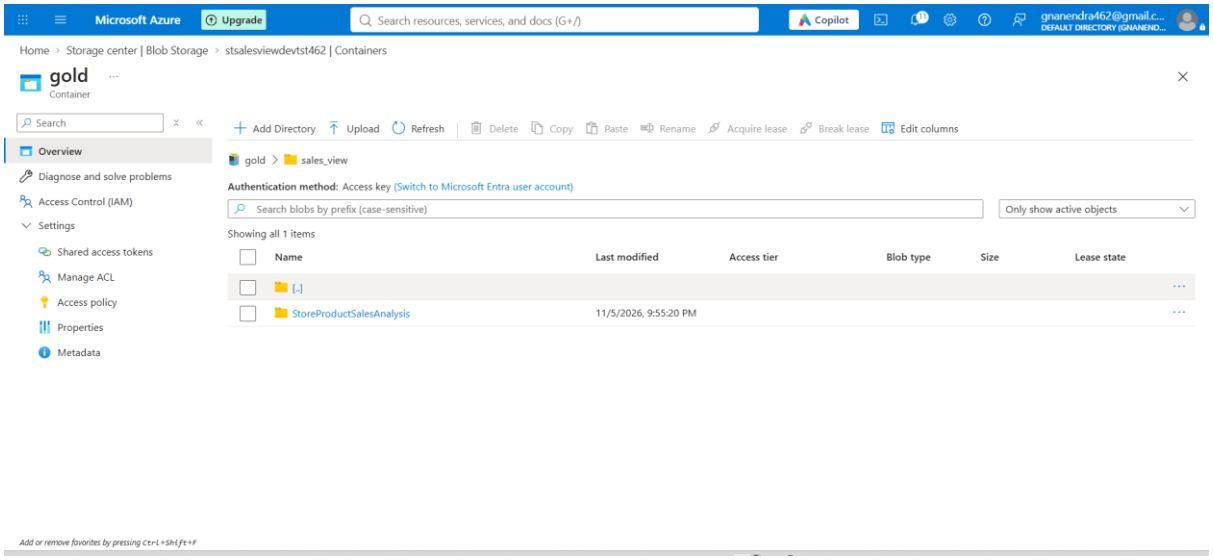
- using product and store table get the below data store_id,store_name,location,manager_name,product_name,product_code,description,category_id,price,stock_quantity,supplier_id,product_created_at,product_updated_at,image_url,weight,expiry_date,is_active,tax_rate.
- Read the delta table (using UDF functions)
- Using the above data & customer_sales and get the below data OrderDate,Category,City,CustomerID,OrderID,Product ID, Profit,Region, Sales,Segment,ShipDate,ShipMode,latitude,longitude,store_name,location,manager_name,product_name,price,stock_quantity,image_url
- Write based on overwrite (table_name : StoreProductSalesAnalysis)(in gold layer path is gold/sales_view/tablename/{delta pearquet}

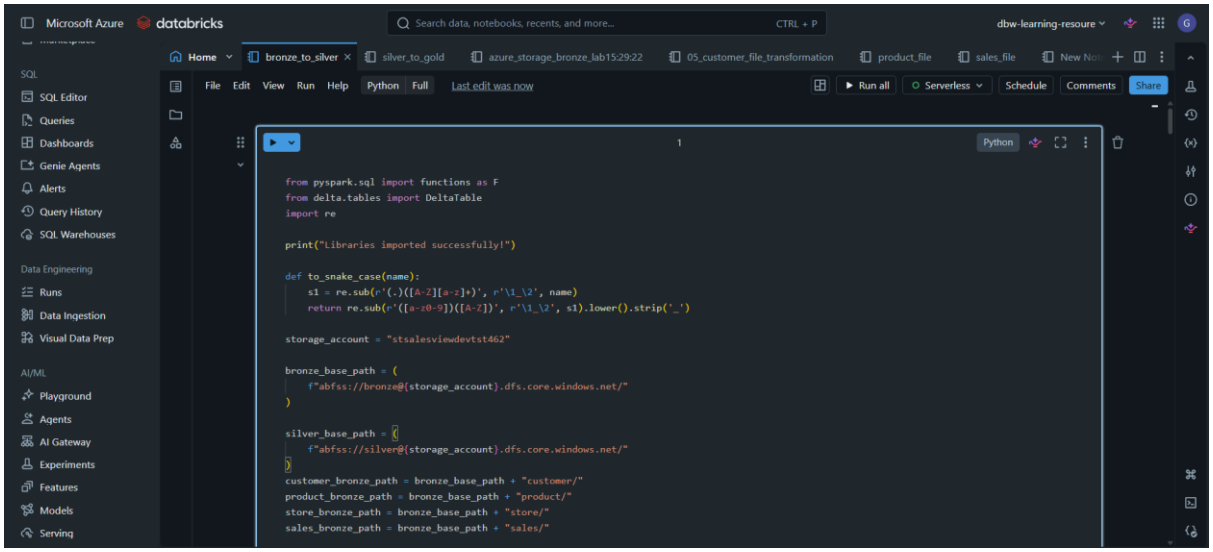

Microsoft Azure
databricks
Search data, notebooks, recents, and more... CTRL + P
dbw-learning-resoure
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Discover
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie Agents
Alerts
Query History
SQL Warehouses
Data Engineering
Runs
Data Ingestion
Visual Data Prep
AI/ML
Home
adls_gen2_connection_test 2026-10-29 12:12:56
New Notebook 2026-10-29 12:23:19
Python
Full
Last edit was now
Just now (1s)
1
Python
Azure Storage Account details
storage_account = "stsalesviewdevtst462"
Silver layer paths
product_path = "abfss://silver@" + storage_account + ".dfs.core.windows.net/sales_view/product"
store_path = "abfss://silver@" + storage_account + ".dfs.core.windows.net/sales_view/store"
sales_path = "abfss://silver@" + storage_account + ".dfs.core.windows.net/sales_view/customer_sales"
Gold layer path
gold_path = "abfss://gold@" + storage_account + ".dfs.core.windows.net/sales_view/StoreProductSalesAnalysis"
print("Product Path:", product_path)
print("Store Path:", store_path)
print("Customer Sales Path:", sales_path)
print("Gold Path:", gold_path)
Product Path: abfss://silver@stsalesviewdevtst462.dfs.core.windows.net/sales_view/product
Store Path: abfss://silver@stsalesviewdevtst462.dfs.core.windows.net/sales_view/store
Customer Sales Path: abfss://silver@stsalesviewdevtst462.dfs.core.windows.net/sales_vie


Microsoft Azure
databricks
Search data, notebooks, recents, and more... CTRL + P
dbw-learning-resoure
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Discover
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie Agents
Alerts
Query History
SQL Warehouses
Home
adls_gen2_connection_test 2026-10-29 12:12:56
New Notebook 2026-10-29 12:23:19
Python
Full
Last edit was now
Diagnose error
Debug
Genie Code Quick Fix: ON
Just now (<1s)
3
Python
storage_account = "stsalesviewdevtst462"
config_name = (
f"fs.azure.account.key.{storage_account}.dfs.core.windows.net"
)
print("Storage configuration name:", config_name)
print("Configuration exists:", config_name in spark.conf.getAll)
Storage configuration name: fs.azure.account.key.stsalesviewdevtst462.dfs.core.windows.net
Configuration exists: False

So, at the end below are things you need to have.







ADF and Azure Databricks-Practical Usw Case Documentation

1.What is Data Profiling and How Is it Done?

Data Profiling is the process of examining, analyzing, and understanding a dataset to identify its quality, structure, patterns, and problems before using it for data processing or analytics.

Types of Data Profiling:

1. Structure Profiling

Checks the structure of data, such as column names, data types, number of rows and columns.

- Column names
- Data types
- Schema validation

2. Content Profiling

Checks the actual values stored in the dataset.

- Null values
- Duplicate records
- Invalid values
- Minimum and maximum values

3. Relationship Profiling

Checks relationships between columns and datasets.

- Primary key validation
- Foreign key validation
- Relationships between tables
- Referential integrity

How is Data Profiling Performed?

Data Profiling Process

1. **Load the source data**

   Read the CSV, JSON or Parquet files from the Bronze layer.

2. **Check the schema**

   Identify column names, data types and total rows and columns.

3. **Check missing values**

   Find null or empty values in each column.

4. **Check duplicate records**

   Identify repeated records and duplicate primary keys.

5. **Analyze distinct values**

   Understand unique values in columns such as Gender, City, Category and Region.

6. **Perform statistical analysis**

   Calculate minimum, maximum, average, count and standard deviation for numeric columns.

7. **Validate data quality**

   Check invalid dates, incorrect email formats, negative prices and inconsistent values.

8. **Generate a profiling report**

   Summarize the findings and decide which records need cleaning or validation.

2.Data Profiling Steps Performed for the Provided Data?

The assignment contains four source datasets:

- Customer
- Product
- Store
- Sales

These datasets were stored in Azure Data Lake Storage Gen2 and processed using Azure Data Factory and Azure Databricks.

2.1 Source Dataset Analysis

The initial source datasets were inspected to understand their structure, columns, data types, and record counts.

Dataset	Number of Records	Profiling Focus
Customer	29	Customer IDs, names, email addresses, gender, joining dates, and expenditure
Product	To be verified	Product IDs, product names, category IDs, prices, stock quantities, and date columns
Store	To be verified	Store IDs, store names, locations, manager details, email addresses, and date columns
Sales	To be verified	Order IDs, customer IDs, product IDs, sales, profit, order dates, and shipping details

2.2 Customer Dataset Profiling

The Customer dataset was analyzed to understand customer information and expenditure details.

The following profiling checks were considered:

1. Checked the total number of records and columns.
2. Inspected column names and data types.
3. Checked customer IDs for duplicate records.
4. Identified NULL or missing values.
5. Examined customer names and email addresses.
6. Checked the available gender values.
7. Inspected the joining date format.
8. Analyzed the `spent` column to understand customer expenditure.

2.3 Product Dataset Profiling

The Product dataset was analyzed to understand product information, categories, and pricing details.

1. Checked the total number of records and columns.
2. Inspected column names and data types.
3. Checked `product_id` for duplicate records.
4. Identified NULL or missing values.
5. Examined product names and product codes.
6. Checked distinct `category_id` values.
7. Analyzed product prices and stock quantities.
8. Inspected product date columns.

2.4 Store Dataset Profiling

The Store dataset was analyzed to understand store details and location information.

The following profiling checks were considered:

1. Checked the total number of records and columns.
2. Inspected column names and data types.
3. Checked `store_id` for duplicate records.
4. Identified NULL or missing values.
5. Examined store names, locations, and manager details.
6. Checked store email addresses and email domains.
7. Inspected `created_at` and `updated_at` date formats.

2.5 Sales Dataset Profiling

The Sales dataset was analyzed to understand order transactions and sales-related information.

The following profiling checks were considered:

1. Checked the total number of records and columns.
2. Inspected column names and data types.
3. Checked order records for duplicates using the appropriate transaction key.
4. Identified NULL or missing values.
5. Examined customer IDs, product IDs, and store IDs.
6. Analyzed sales and profit values.
7. Inspected order dates and shipping dates.

3. Complete the Above Practical Use Case Given

3.1 Project Objective

The objective of this practical use case is to build an end-to-end data engineering pipeline using Azure services.

The pipeline processes four datasets:

1. Customer
2. Product
3. Store
4. Sales

The source files are ingested using Azure Data Factory and stored in Azure Data Lake Storage Gen2. Azure Databricks is used to transform and process the data using PySpark. Delta Lake is used to maintain the transformed data through upsert and overwrite operations.

3.2 ADLS Container and Folder Creation

Initially, a storage account was used to create the required containers and folders.

The following containers were created:

- Bronze
- Silver
- Gold

The folder structure was organized as follows:

Bronze Container

- customer
- product
- store
- sales

Silver Container

- sales_view/customer
- sales_view/product
- sales_view/store
- sales_view/customer_sales

Gold Container

- sales_view/StoreProductSalesAnalysis

The Bronze layer stores raw source files, the Silver layer stores cleaned and transformed data, and the Gold layer stores the final analytical dataset.

3.3 Azure Data Factory Pipeline Creation

Azure Data Factory was used to automate the ingestion of source files from ADLS.

The following activities were considered:

1. Created an Azure Data Factory pipeline.
2. Configured pipeline parameters to make the pipeline dynamic.
3. Used file metadata to identify the latest modified files in the source folders.
4. Configured source and destination datasets.
5. Used the Copy Data activity to transfer files into the Bronze layer.
6. Passed parameter values dynamically to process different folders and files.
7. Executed the pipeline and verified the copied files in ADLS.